LAB 7 CHE655

Name: AADITYA AMLAN PANDA
Roll: 220007

INTRODUCTION

The problem statement asks us to obtain an optimisation problem of the linear equation Ax = b to find the most suitable x using Newton's method and Levenberg Marquardt algorithm. The code can be seen in the appendix section.

METHODOLOGY

- **NEWTON'S METHOD:-**
  - Step 1: First the problem is made into an error optimisation problem: -
    - minimise $F(x) = 0.5||Ax-b||^2$ w.r.t. x.
  - Step 2: The matrix function of the error looks like F(x) = 0.5(Ax-b)'(Ax-b) that becomes: -
    - F(x) = 0.5*(x'A'Ax – 2b'Ax + b'b) = 0.5x'Sx + c'x + 0.5b'b, where S = A'A and c = -b'A
  - Step 3: The gradient is calculated using matrix derivative methods as ∇F = A'(Ax-b).
  - Step 4: The Hessian is calculated $H = \nabla^2 F = A'A$
  - Step 5: The equation $H(x_k)\Delta x_k = -\nabla F(x_k)$ is solved using Gaussian Elimination to obtain Δx_k
  - Step 6: x is updated using $x_{k+1} = x_k + \Delta x_k$
  - Step 7: The above steps are repeated until $x_{k+1} - x_k < 1e\text{-}6$

- **LEVENBERG MARQUARDT ALGORITHM:-**
  - Step 1: First the problem is made into an error optimisation problem: -
    - minimise $F(x) = ||Ax-b||^2$ w.r.t. x.
  - Step 2: The matrix function of the error looks like F(x) = (Ax-b)'(Ax-b)
  - Step 3: The Jacobian is calculated as $J = \frac{\partial y}{\partial p}$ using matrix derivative methods where y = Ax here . This gives Since $y = \sum A_{ij} * x_j$, $\frac{\partial y}{\partial x_j}$ *gives* A_{ij}. Hence J= A
  - Step 4: The gradient is calculated using matrix derivative methods as $\nabla F = 2J'(y - \hat{y}(p)) = 2A'(b - Ax)$
  - Step 5: The Hessian is approximated as $H = \nabla^2 F = 2J'J = 2A'A$
  - Step 6: In the LM step, the equation $(H(x_k) + \lambda I)\Delta x_k = -\nabla F(x_k)$, is approximated to Newton (as per Strang) $(J'J + \lambda I)\Delta x_k = J'(b - Ax)$. This is solved usign solved using Gaussian Elimination to obtain Δx_k
  - Step 7: x is updated using $x_{k+1} = x_k + \Delta x_k$
  - Step 8: The value of λ (damping factor) is decided based on whether the solution is far away from minima value. This is known via keeping a track of current error = $x_{k+1} - x_k$ and previous error = $x_k - x_{k-1}$ **.** If previous error is larger, it means we are approaching closer to the minima, λ is updated using $\lambda = \lambda/10$. If previous error is smaller, it means we are moving away from the minima, λ is updated using $\lambda = \lambda*10$
  - Step 9: The above steps are repeated until $x_{k+1} - x_k < 1e\text{-}6$

RESULTS

Previous Methods:-

Using Gaussian Elimination X = [[1.37533631] [-5.23908314]]
Using Inversion of AtA X = [[1.37533631] [-5.23908314]]

Using Full Rank SVD X = [[1.37533631] [-5.23908314]]
Using Partial Rank SVD X = [[1.37533631] [-5.23908314]]
Using lstsq for verification X = [[1.37533631] [-5.23908314]]

For Newton's Method
Using Newton's Method X = [[1.37533631] [-5.23908314]]
Number of iterations for convergence in Newton's method = 2

For LM:-
Damping factor = 0.001 (small damping factor), X = [[1.37321165] [-5.23542899]]
Using Levenberg-Marquardt Method X = [[1.37533631] [-5.23908314]]
For an initial damping factor of λ = 0.001
Number of iterations for convergence in Levenberg-Marquardt method = 3
Final value of damping factor = 1e-05

n = 1.3753363081627297
k = 0.005305118661095586

OBSERVATIONS

1. It can be observed that the values of x obtained using Newton's Method and LM are equal to the ones obtained in the previous lab using different methods and the value of n and k also match the values mentioned in Levenspiel.
2. For low values of λ, it can be observed results are much similar to that of Newton's
   Using Levenberg-Marquardt Method X = [[1.37533631] [-5.23908314]]
   Number of iterations for convergence = 5
   Final value of damping factor = 0.01
3. For an iterative LM process when damping factor is adjusted more often with the error tracking, the number of iterations decrease with the results being similar again.
4. Relative to Newton's, for this particular problem, it can be said that LM is slower as per Big O time complexity notation. Hence Newton's should be the best strategy to adopt. If LM has to be used, the iterative LM provides a much accurate nearer answer.

APPENDIX

```
import numpy as np
import math
import pandas as pd
import matplotlib.pyplot as plt

def find_col_norm(V, pos, n):
s = 0.0
for i in range(n):
s += V[i][pos] * V[i][pos]
return math.sqrt(s)

def find_vec_norm(V, n):
s = 0.0
for i in range(n):
s += V[i] * V[i]
return math.sqrt(s)

def find_dot_vec(A, B, n):
s = 0.0
for i in range(n):
s += A[i] * B[i]
return s

def find_transpose(A, n, m):
Atrp = np.zeros((m, n))
for i in range(n):
for j in range(m):
Atrp[j][i] = A[i][j]
return Atrp

def matrix_multiply(A1, A2, n, t, m):
Ares = np.zeros((n, m))
for i in range(n):
for j in range(m):
for k in range(t):
Ares[i][j] += A1[i][k] * A2[k][j]
return Ares

def uv_prod(U, V, n, m):
res = np.zeros((n, m))
for i in range(n):
for j in range(m):
res[i][j] = U[i] * V[j]
return res

def matrix_sum(A, B, n, m, flag):
res = np.zeros((n, m))
for i in range(n):
```

```
for j in range(m):
if flag ==0:
res[i][j] = A[i][j] + B[i][j]
else:
res[i][j] = A[i][j] - B[i][j]
return res

def SVD(A):
n,m = A.shape
eps = 1e-12

if n >= m:
Atrp = find_transpose(A, n, m)
A_curr = matrix_multiply(Atrp, A, m, n, m)
N = m

Q_final = np.eye(N)

for _ in range(30):
Q = np.zeros((N, N))
R = np.zeros((N, N))

for j in range(N):
v = np.zeros(N)
for i in range(N):
v[i] = A_curr[i][j]

for i in range(j):
qi = np.zeros(N)
for k in range(N):
qi[k] = Q[k][i]
R[i][j] = find_dot_vec(qi, A_curr[:, j], N)
for k in range(N):
v[k] -= R[i][j] * qi[k]

R[j][j] = find_vec_norm(v, N)
if R[j][j] < eps:
continue

for k in range(N):
Q[k][j] = v[k] / R[j][j]

A_curr = matrix_multiply(R, Q, N, N, N)
Q_final = matrix_multiply(Q_final, Q, N, N, N)

V = Q_final
sigma = np.zeros((N, N))
sigma_inv = np.zeros((N, N))

for i in range(N):
val = A_curr[i][i]
if val < 0:
val = 0
sigma[i][i] = math.sqrt(val)
if sigma[i][i] > eps:
sigma_inv[i][i] = 1.0 / sigma[i][i]

AV = matrix_multiply(A, V, n, m, N)
U = matrix_multiply(AV, sigma_inv, n, N, N)

for j in range(N):
norm = find_col_norm(U, j, n)
if norm > eps:
for i in range(n):
U[i][j] /= norm

else:
Atrp = find_transpose(A, n, m)
A_curr = matrix_multiply(A, Atrp, n, m, n)
N = n

Q_final = np.eye(N)

for _ in range(30):
Q = np.zeros((N, N))
R = np.zeros((N, N))

for j in range(N):
v = np.zeros(N)
for i in range(N):
v[i] = A_curr[i][j]

for i in range(j):
qi = np.zeros(N)
for k in range(N):
qi[k] = Q[k][i]
R[i][j] = find_dot_vec(qi, A_curr[:, j], N)
for k in range(N):
v[k] -= R[i][j] * qi[k]

R[j][j] = find_vec_norm(v, N)
if R[j][j] < eps:
continue

for k in range(N):
Q[k][j] = v[k] / R[j][j]

A_curr = matrix_multiply(R, Q, N, N, N)
Q_final = matrix_multiply(Q_final, Q, N, N, N)

U = Q_final
sigma = np.zeros((N, N))
sigma_inv = np.zeros((N, N))

for i in range(N):
val = A_curr[i][i]
if val < 0:
val = 0
sigma[i][i] = math.sqrt(val)
if sigma[i][i] > eps:
sigma_inv[i][i] = 1.0 / sigma[i][i]

Ut = find_transpose(U, N, N)
UtA = matrix_multiply(Ut, A, N, n, m)
Vt = matrix_multiply(sigma_inv, UtA, N, N, m)
V = find_transpose(Vt, N, m)

return U, sigma, V


def truncate_A(U,sigma,V,n,m):
k = int(input("Enter k for truncated summation: "))
k = min(k, min(n, m))

A_k = np.zeros((n, m))
for i in range(k):
uv = uv_prod(U[:, i], V[:, i], n, m)
for r in range(n):
for c in range(m):
uv[r][c] *= sigma[i][i]
A_k = matrix_sum(A_k, uv, n, m, 0)

return A_k
```

```
def print_SVD(U,sigma,V,A_k):
print("\nU =")
print(U)

print("\nSigma =")
print(sigma)

print("\nV =")
print(V)

print("\nReconstructed A using k terms =")
print(A_k)

def gauss_elim(A,B):
n,m = A.shape
A1 = A.copy()
B1 = B.copy()
for i in range(n-1):
pivot = A1[i][i]
for t in range(i+1,n):
targ = A1[t][i]
for j in range(m):
A1[t][j] = A1[t][j] - ((A1[i][j]/pivot) * targ)
B1[t] = B1[t] - (B1[i]/pivot) * targ

res = np.zeros((n,1))
j= n-2
res[n-1] = B1[n-1]/A1[n-1][n-1]
for i in range(j,-1,-1):
loc = m-1
sum = 0.0
for loc in range(i+1,m):
sum += A1[i][loc] * res[loc]
loc -= 1
res[i] = (B1[i]-sum)/A1[i][i]

return res

def invert_2by2(A):
adj = np.zeros((2,2))
inv = np.zeros((2,2))
det = A[0][0] * A[1][1] - A[0][1]*A[1][0]
adj[0][0] = A[1][1]
adj[1][1] = A[0][0]
adj[0][1] = -A[0][1]
adj[1][0] = -A[1][0]

inv[0][0] = adj[0][0]/det
inv[1][1] = adj[1][1]/det
inv[0][1] = adj[0][1]/det
inv[1][0] = adj[1][0]/det

return inv

def find_derivative(A,b,x):
n,m = A.shape
res = np.zeros((m,1))
Ax = matrix_multiply(A, x, n, m, 1)
Ax_b = matrix_sum(Ax, b, n, 1, 1)
Atrp = find_transpose(A, n, m)
res = matrix_multiply(Atrp, Ax_b, m, n, 1)
return res
def hessian_matrix(A):
n,m = A.shape
H = np.zeros((m,m))
Atrp = find_transpose(A, n, m)
H = matrix_multiply(Atrp, A, m, n, m)
return H

dat = pd.read_csv("data.csv", header=None).values


T = dat[:,0:1]
Ca = dat[:,1:2]
rate = dat[:,2:3]
l= rate.shape[0]
B= np.zeros((l,1))
A= np.ones((l,2))


for i in range(l):
B[i] = math.log(rate[i])
A[i][0] = math.log(Ca[i])

n,m = A.shape

## METHOD 1
Atrp = find_transpose(A, n, m)
AtA = matrix_multiply(Atrp, A, m, n, m)
Atb = matrix_multiply(Atrp, B, m, n, 1)

res_mth_1 = gauss_elim(AtA, Atb)
print("\nUsing Gaussian Elimination X =")
print(res_mth_1)

## METHOD 1.1
AtA_inv = invert_2by2(AtA)
res_mth_1_1 = matrix_multiply(AtA_inv, Atb, 2, 2, 1)
print("\nUsing Inversion of AtA X =")
print(res_mth_1_1)

## METHOD 2
U,sigma,V = SVD(A)

sigmainv = invert_2by2(sigma)
Ut = find_transpose(U, n, m)
Utb = matrix_multiply(Ut, B, m, n, 1)
sigmainvUtb = matrix_multiply(sigmainv, Utb, 2, 2, 1)

res_mth_2 = matrix_multiply(V, sigmainvUtb, 2, 2, 1)
print("\nUsing Full Rank SVD X =")
print(res_mth_2)

## METHOD 2.1
y = np.zeros((m,1))
for i in range(m):
temp = matrix_multiply(Ut[i:i+1,:], B, 1, n, 1)
y[i] = temp[0]/sigma[i][i]

res_mth_2_1 = matrix_multiply(V, y, m, m, 1)
print("\nUsing Partial Rank SVD X =")
print(res_mth_2_1)

## Verification using lstsq
X, residuals, rank, s = np.linalg.lstsq(A, B, rcond=None)
print("\nUsing lstsq for verification X =")
print(X)

## METHOD 3: NEWTONS METHOD
H = hessian_matrix(A)
x_new = np.zeros((m,1)) # Initial guess
x = np.ones((m,1)) # for first iteration
eps = 1e-6
```

```
i=0
while find_vec_norm(matrix_sum(x, x_new, m, 1, 1), m) >
eps:
x = x_new
grad = find_derivative(A, B, x)
delta_x = gauss_elim(H, -grad)
x_new = matrix_sum(x, delta_x, m, 1, 0)
i += 1

res_mth_3 = x_new
print("\nUsing Newton's Method X =")
print(res_mth_3)
print("\nNumber of iterations for convergence in
Newton's method =")
print(i)

## METHOD 3.1: LEVENBERG-MARQUARDT
J = A # Jacobian
H = 2*hessian_matrix(J)
x_new = np.zeros((m,1)) # Initial guess
x = np.ones((m,1)) # for first iteration
eps = 1e-6
i=0
df = 0.01
mult = 10.0
I = np.eye(m)
err_curr = 1e10
err_prev = 1e20
while err_curr > eps:
if err_curr < err_prev:
df = df/mult
else:
df = df*mult
x = x_new
grad = find_derivative(A, B, x)
dfI = df * I
H_dfI = matrix_sum(hessian_matrix(J), dfI, m, m, 0)
delta_x = gauss_elim(H_dfI, -grad)
x_new = matrix_sum(x, delta_x, m, 1, 0)
if df == 0.001:
print("\nDamping factor = 0.001 (small damping factor), X
=")
print(x_new)
err_prev = err_curr
err_curr = find_vec_norm(matrix_sum(x, x_new, m, 1, 1), m)
i += 1

res_mth_3_1 = x_new
print("\nUsing Levenberg-Marquardt Method X =")
print(res_mth_3_1)
print("\nNumber of iterations for convergence in
Levenberg-Marquardt method =")
print(i)
print("\nFinal value of damping factor =")
print(df)

final_res = res_mth_3_1
n = final_res[0][0]
k = math.exp(final_res[1])

print("\nn =")
print(n)
print("\nk =")
print(k)
```